A Formal Security Analysis of a Simplified MACsec Key Agreement Protocol Using Tamarin

Halil İbrahim Kaplan
Cryptanalysis Laboratory, TÜBİTAK BİLGEM
Kocaeli, Turkey
halil.kaplan@tubitak.gov.tr

***Abstract*—MACsec Key Agreement (MKA) is the IEEE 802.1X key-management protocol used to establish and maintain Secure Associations for MACsec deployments. Although MKA is widely deployed, machine-checked analyses of its core key-agreement logic remain scarce. This paper presents a formal analysis of a simplified two-party MKA exchange using the Tamarin prover. We model the initial session establishment and a subsequent rekey round, and verify secrecy, authentication, agreement, ordering, and freshness properties. The analysis confirms these guarantees under a Dolev–Yao adversary when the pre-shared Connectivity Association Key (CAK) is not compromised. We also identify a structural weakness: a malicious or compromised Key Server can inject an arbitrary Secure Association Key (SAK) that the Server accepts. This finding clarifies the trust assumptions of MKA and motivates additional verification or binding mechanisms for partially trusted deployments.**

I. Introduction

The increasing deployment of Ethernet-based infrastructures has weakened the assumption that the data-link layer is inherently trustworthy. In these settings, link-layer traffic is exposed to adversarial conditions, and cryptographic protection is needed to preserve confidentiality, integrity, and replay resistance. The IEEE 802.1AE MACsec standard [1] addresses this need by defining hop-by-hop protection for Ethernet frames, while the MACsec Key Agreement protocol (MKA) [2] manages the establishment and maintenance of the cryptographic state used by MACsec.

MKA operates within a Connectivity Association (CA), where an elected Key Server derives fresh Secure Association Keys (SAKs) and distributes them to participating peers. The protocol relies on a pre-shared Connectivity Association Key (CAK), from which derived keys such as the Key Encryption Key (KEK) and the Integrity Check Value Key (ICK) are computed. Message Numbers (MNs) and Key Numbers (KNs) are used to track replay protection and key-generation epochs, including periodic rekeying. Despite the practical importance of MKA, formal analyses of its key-agreement logic remain limited. Existing work has largely focused on MACsec data-plane mechanisms or performance aspects rather than on machine-checked verification of the protocol's core security guarantees.

This paper addresses that gap by presenting a formal analysis of a simplified two-party MKA exchange using the Tamarin prover. Our model captures the initial session establishment and a subsequent rekey round, and we verify essential security properties under a standard Dolev–Yao adversary model. The analysis confirms the expected guarantees when the CAK remains uncompromised, while also revealing a structural weakness in the protocol's acceptance logic.

a) Contributions.: This work makes three contributions. First, it constructs a Tamarin model of a two-party MKA exchange that captures the initial session and a rekey round, including the monotonic MN/KN counters. Second, it formalizes and verifies core security properties such as executability, agreement, authentication, secrecy, ordering, and freshness. Third, it identifies and demonstrates a structural weakness in the protocol's acceptance logic: a malicious or compromised Key Server that possesses the CAK can inject an arbitrary SAK that the Server accepts.

b) Organization.: Section II explains MKA and its lifecycle. Section III introduces the formal analysis and the Tamarin prover. Section IV presents our formal model and the protocol flow in Figure 1. Section V reports the verification results, and Section VI concludes the paper.

II. MKA Overview

A. IEEE 802.1X MACsec Key Agreement (MKA)

IEEE 802.1AE (MACsec) [1] provides confidentiality, integrity, and replay protection for Ethernet frames on a per-hop basis. The associated key-management protocol, defined in IEEE 802.1X [2], is MACsec Key Agreement (MKA). MKA operates over the EAP over LAN (EAPoL) control plane and supports the following lifecycle:

1) **Connectivity Association setup.** Participants share a *Connectivity Association Key* (CAK) and a Connectivity Association Key Name (CKN), either through pre-provisioning or through an EAP exchange. A Connectivity Association (CA) is formed among peers that share the same CAK.
2) **Key derivation.** Each participant independently derives a *Key Encryption Key* (KEK) and an *Integrity Check Value Key* (ICK) from the CAK using a key derivation function (KDF). These keys are then used to protect MKA PDUs (MKAPDUs).
3) **Key Server election.** A Key Server is elected among CA members according to the protocol's priority and tie-breaking rules. The elected Key Server is responsible for generating SAKs.

4) **SAK distribution.** The Key Server generates a fresh SAK, encrypts it with the KEK, appends an Integrity Check Value (ICV) computed with the ICK, and distributes the resulting MKAPDU to the other members. Each receiver verifies the ICV, decrypts the SAK, and acknowledges receipt.
5) **Rekey.** SAKs are periodically refreshed, either because of traffic thresholds or administrative policy. Rekeying follows the same structure as the initial distribution, but with incremented Key Number (KN) and Message Number (MN) values.

The protocol flow shown in Figure 1 illustrates the simplified two-party MKA exchange between a Key Server and a Server that we analyse in this paper, using the exact variable names and state facts of `MKA_v10.spthy`.

In the model, the *Message Number* (MN) is a per-participant, monotonically increasing counter produced by `genMN` and advanced via `count`: $MN\_KS1 = \texttt{count}(MN\_KS)$, $MN\_KS2 = \texttt{count}(MN\_KS1)$, and analogously for the Server side. MNs protect against MKAPDU replay. The *Key Number* (KN) identifies each key-generation epoch and advances at each rekey: $KN1 = \texttt{count}(\tilde{kn})$, where $\tilde{kn}$ is the fresh KN from the initial session. The state facts `KSSTATE0/1/2` and `SSTATE0/1` encode the protocol state machine and enforce strict sequencing between the initial session and the rekey round.

III. Formal Analysis and Tamarin Prover

The Tamarin Prover [3], [4] is a widely used tool for the automated symbolic verification of security protocols. It operates in the Dolev–Yao adversary model [5]: the adversary controls the network (can intercept, replay, modify, and inject messages) but cannot break cryptographic primitives (for example, cannot invert a hash, decrypt without the key, or forge a MAC).

a) Modelling language.: Tamarin protocols are specified as *multiset rewriting rules* over a set of *facts*. Each rule has the form

$$[\text{premise facts}] \xrightarrow{[\text{action facts}]} [\text{conclusion facts}]$$

The premise specifies what facts must be present in the current system state, action facts annotate the rule firing with observable events, and the conclusion specifies what facts are produced. The special facts `Fr(x)` generate fresh nonces, `In(m)` receives a network message, and `Out(m)` sends one. Persistent facts (prefixed with `!`) survive consumption.

b) Equational theories.: Tamarin supports equational theories that model algebraic properties of cryptographic operations. For instance, `sdec(senc(m,k),k) = m` captures symmetric encryption/decryption, and `macverify(mac(M,K),M,K) = mactrue` captures MAC verification. The tool reasons about an arbitrary Dolev-Yao adversary that can apply these equations to derive new messages.

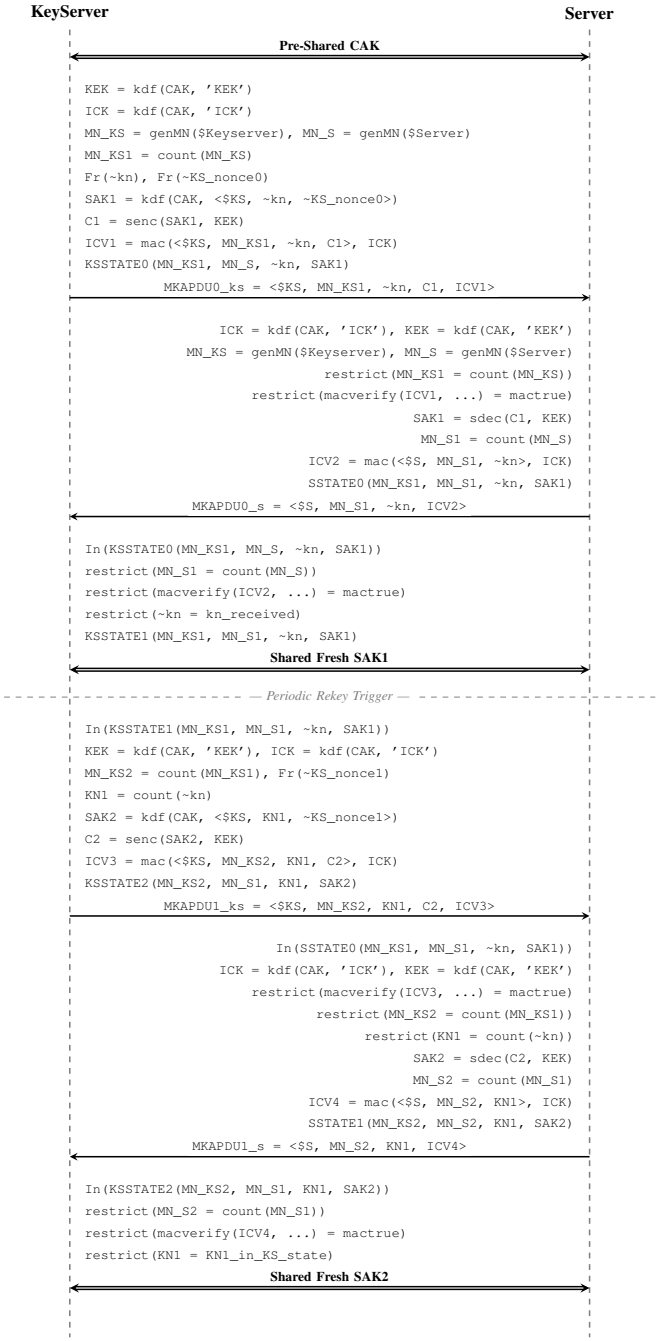

Fig. 1. Simplified two-party MKA protocol flow (initial session and rekey round) as specified in `MKA_v10.spthy`. $KS = KeyServer identity, $S = Server identity, MN = Message Number counter, KN = Key Number counter.

c) Restrictions and lemmas.: Restrictions (formerly called axioms) constrain the set of valid execution traces, allowing the modeller to express protocol-specific validity conditions (e.g., "a MAC must verify"). Lemmas specify security properties in a fragment of first-order logic with time annotations:

- `all-traces` lemmas express universal properties (safety, authentication, secrecy).
- `exists-trace` lemmas assert executability or demonstrate attack traces.

Tamarin employs a constraint-solving procedure with manual or automatic heuristics to discharge lemmas. Its soundness and completeness guarantees hold relative to the modelled equational theory.

d) Rule syntax and a concrete MKA example.: A Tamarin rule is declared as follows:

```
rule Setup_CAK:
  [ Fr(~cak) ]
  --[ CAKSetup(~cak) ]->
  [ !SharedCAK(~cak) ]
```

The left-hand side (before `->`) lists premise facts; the right-hand side lists conclusion facts. For the MKA protocol, the initial session is captured by rules such as `Initial_KeyServer_Send`, `Initial_Server_Receive`, and `Initial_KeyServer_Confirm`, each encoding the state transitions depicted in Figure 1.

e) Monotonic counters and the `natural-numbers` builtin.: In our model, counters (MN and KN) are incremented using the `count(x)` function. To ensure sound reasoning about monotonicity, the `natural-numbers` builtin must be declared in the header of the `.spthy` file:

```
theory MKA_v10:
  builtins: natural-numbers

begin
...
end
```

Without this builtin, Tamarin treats `count` as a generic function and may allow non-intuitive behaviour such as `count(x) = x` or `count(x) < x`, potentially invalidating proofs about session ordering.

f) Debugging and reproducibility.: When a lemma fails, Tamarin produces a concrete counterexample trace. This trace can be exported to LaTeX via the `-trace` option, which generates a `trace.tex` file that can be directly included in the paper. For reproducibility, all proof outputs—including `summary.pdf`, `trace.tex`, and the exact command-line invocation—are archived alongside the model in the repository.

IV. Our Formal Model

A. Cryptographic Primitives and Equational Theory

Our Tamarin model is built on the following builtins and functions:

- **Symmetric encryption/decryption:** `senc`/`sdec` satisfying `sdec(senc(m,k),k) = m`.
- **Key derivation:** `kdf(key,label)` is modelled as an uninterpreted two-argument function; no algebraic identities are assumed, which captures the idealised one-way property of a KDF.
- **Message authentication:** `mac(M,K)` and `macverify(mac(M,K),M,K) = mactrue`; the equation ensures that the adversary can only produce a valid ICV if it knows the ICK.
- **Monotone counters:** `count(x)` models counter advancement and is used for both MN and KN progression. The `natural-numbers` builtin is enabled to provide a sound treatment of this function.
- **Message-number generation:** `genMN($Party)` is an uninterpreted function that assigns a fresh initial MN to a named party, ensuring per-party counter initialisation.

B. Protocol Participants and Long-Term State

a) CAK setup.: The rule `Setup_CAK` generates a fresh CAK $\tilde{cak}$ and stores it as a persistent shared fact `!SharedCAK(~cak)`, available to both parties.

b) Derived keys.: Both participants independently compute

$$kek = \texttt{kdf}(cak, \texttt{'KEK'}), \quad ick = \texttt{kdf}(cak, \texttt{'ICK'})$$

at the beginning of each protocol step, since both parties share the same CAK.

C. Initial Session Rules

a) Initial_KeyServer_Send.: The Key Server:

1) Retrieves `!SharedCAK(~cak)`.
2) Generates a fresh Key Number $\tilde{kn}$ and a fresh nonce $\widetilde{KS\_nonce0}$.
3) Derives $sak1 = \texttt{kdf}(cak, \langle \$Keyserver, \tilde{kn}, \widetilde{KS\_nonce0} \rangle)$.
4) Computes $c1 = \texttt{senc}(sak1, kek)$.
5) Computes $icv1 = \texttt{mac}(\langle \$Keyserver, mn\_ks1, \tilde{kn}, c1 \rangle, ick)$.
6) Sends $\langle \$Keyserver, mn\_ks1, \tilde{kn}, c1, icv1 \rangle$ and stores `KSSTATE0`.

The action fact `SentInitialSAK` is recorded for use in the authentication lemmas.

b) Initial_Server_Receive.: The Server:

1) Receives the MKAPDU from the network.
2) Retrieves `!SharedCAK(~cak)` and verifies $icv1$ using a `_restrict` action: `macverify(icv1, ..., ick) = mactrue`.
3) Restricts the received MN to match the expected counter: $mn\_ks1 = \texttt{count}(mn\_ks)$.
4) Decrypts the payload: $sak1 = \texttt{sdec}(c1, kek)$.
5) Computes and sends its acknowledgement ICV2.
6) Records `ServerAcceptedSAK` and stores `SSTATE0`.

c) Initial_KeyServer_Confirm.: The Key Server receives the Server's ACK, verifies ICV2, checks that the received MN matches the expected counter, and transitions to `KSSTATE1`, recording `InitialSessionCompleted`.

D. Rekey Rules

The rekey phase mirrors the initial phase structurally, but it reads the previous state (`KSSTATE1`/`SSTATE0`) to enforce monotonicity. Specifically:

- $mn\_ks2 = \texttt{count}(mn\_ks1)$ and $kn1 = \texttt{count}(\tilde{kn})$ ensure both counters advance.
- A fresh nonce KS_nonce1 ensures SAK2 is independent of SAK1 even for identical KN paths.
- `Rekey_KeyServer_Send` records `RekeyStarted`, `Rekey_Server_Receive` records `RekeyAckPrepared`, and `Rekey_KeyServer_Confirm` records `RekeySessionCompleted`.

E. Adversary Model and Compromise Rules

We model a standard Dolev–Yao adversary augmented with two compromise rules:

- `Reveal_SAK`: exposes an initial SAK from `KSSTATE0`; records `RevealSAK`.
- `Reveal_CAK`: exposes the CAK from `!SharedCAK`; records `RevealCAK`.

These rules allow the formulation of conditional security properties, for example that secrecy holds unless the CAK is revealed, and they capture the forward-secrecy limitations inherent in a pure pre-shared-key protocol.

Additionally, the rule `Malicious_KeyServer_Send` models a Key Server that uses the CAK to construct a well-formed MKAPDU carrying an arbitrary fresh SAK ($\widetilde{fakeSAK}$) instead of a properly KDF-derived one. This rule is used to probe the protocol's acceptance mechanism.

F. Security Properties

We specify twelve security properties as Tamarin lemmas. Table I summarises them. The checked entries show that the protocol is executable and that the main agreement, authentication, secrecy, and ordering properties hold under the stated assumptions. The two existence lemmas are especially important: they confirm that the model admits a valid execution trace, while also exposing the structural acceptance weakness in which a malicious Key Server can inject a non-KDF-derived SAK that the Server accepts.

G. Protocol Executability

Definition 1 (Executability)**.** *The model is* executable *if there exists at least one complete trace in which both* `InitialSessionCompleted` *and* `RekeySessionCompleted` *are observed.*

This is expressed as an `exists-trace` lemma and serves as a sanity check that the model does not over-restrict execution through contradictory restrictions.

H. Agreement Properties

Definition 2 (Initial Agreement)**.** *For every trace in which* `InitialSessionCompleted`$(mn_{ks1}, mn_{s1}, kn, sak1)$ *is observed at time i, there exists a strictly earlier time $j < i$ at which* `SentInitialSAK`$(mn_{ks1}, kn, sak1)$ *was recorded.*

TABLE I
SECURITY LEMMAS VERIFIED AGAINST THE MKA MODEL.

Lemma	Type	Property	Res.
`executable`	exists-trace	Executability	✓
`initial_agreement`	all-traces	Initial agreement	✓
`rekey_agreement`	all-traces	Rekey agreement	✓
`server_authentication_initial`	all-traces	Server aliveness	✓
`server_authentication_rekey`	all-traces	Rekey aliveness	✓
`initial_sak_secrecy`	all-traces	SAK1 secrecy	✓
`rekey_sak_secrecy`	all-traces	SAK2 secrecy	✓
`rekey_cannot_start_before_initial`	all-traces	Ordering	✓
`rekey_after_initial_completion`	all-traces	Ordering	✓
`key_update`	all-traces	SAK1 $\neq$ SAK2	✓
`key_number_progress`	all-traces	KN monotonicity	✓
`fresh_rekey_key`	all-traces	Rekey freshness	✓
`server_only_accepts_kdf_key`	exists-trace	KDF bypass	✓
`accepts_non_kdf_sak`	exists-trace	Malicious SAK	✓

This property captures *non-injective agreement* (aliveness plus agreement on key material) from the Key Server's perspective: the Key Server cannot complete a session with a SAK it never sent.

Definition 3 (Rekey Agreement)**.** *Analogously, for every* `RekeySessionCompleted` *event, there exists an earlier* `RekeyStarted` *event with matching parameters.*

I. Server Authentication

Definition 4 (Server Authentication — Initial)**.** *If* `InitialSessionCompleted` *is observed and the CAK has not been revealed, then* `InitialAckPrepared` *must have been observed earlier by the Server.*

This formalizes the requirement that the Key Server cannot consider a session complete unless the Server has actually processed the MKAPDU and generated a valid MAC acknowledgement, that is, the Server is *alive* and has participated in the exchange.

J. SAK Secrecy

Definition 5 (Initial SAK Secrecy)**.** *If* `InitialSessionCompleted` *is observed, the SAK itself has not been directly revealed (*`RevealSAK`*), and the CAK has not been revealed (*`RevealCAK`*), then the adversary does not know $sak1$ (i.e., $K(sak1)$ is not derivable).*

The conditional structure reflects the inherent trust assumptions of a pre-shared-key protocol: compromise of the CAK transitively compromises all derived keys.

K. Session Ordering and Key Freshness

The lemmas `rekey_cannot_start_before_initial_completion` and `rekey_after_initial_completion` together express that the rekey sub-protocol is strictly sequenced after a successfully completed initial session. The lemma `key_update` asserts $sak1 \neq sak2$, `key_number_progress` asserts $kn1 = \texttt{count}(kn)$, and `fresh_rekey_key` asserts $sak2$ was not present in any initial session — together these

capture the semantic freshness and progression guarantees of the rekey mechanism.

L. Structural Weakness: KDF-Bypass Attack

Definition 6 (Malicious SAK Acceptance)**.** *There exists a trace in which* `MaliciousSAKSent(sak)` *is observed before* `ServerAcceptedSAK(sak)`*, and sak was never output by* `KDFGeneratedSAK`*.*

This exists-trace lemma captures the scenario in which a Key Server holding the CAK injects a fresh but non-KDF-derived SAK into a well-formed MKAPDU. The Server, having no independent means of verifying whether the SAK was derived through the prescribed KDF path or was arbitrary, accepts it. This is a *structural* property of the protocol rather than a cryptographic break.

V. Verification Results

We verified the model with Tamarin and obtained proofs for the main safety and authentication lemmas. The executable lemma succeeds, showing that the modeled protocol can run to completion under the adversary. The agreement and authentication lemmas also hold, indicating that the Key Server and Server agree on the session parameters and that the Server only accepts a session after the expected protocol step has occurred.

The secrecy lemmas hold whenever the CAK is not revealed. In other words, the derived SAKs remain unknown to the adversary unless the shared long-term secret is compromised. The ordering lemmas also succeed, confirming that rekeying cannot occur before an initial session is completed and that the KN/MN counters advance monotonically.

The most interesting result is the existence of an attack trace for the acceptance condition. Tamarin finds a trace in which a malicious Key Server sends a well-formed MKAPDU carrying an arbitrary fresh SAK. Because the MAC is computed with the ICK derived from the shared CAK, the Server accepts the message and records the injected SAK as valid. This is not a break of the underlying cryptographic primitives; rather, it exposes a structural limitation of the protocol's trust model. The Server has no independent proof that the SAK was generated through the prescribed KDF path, so a compromised or malicious Key Server can silently substitute its own value.

A. Related Work

Formal verification has become standard for protocol analysis. Classical examples include Lowe's analysis of Needham–Schroeder with FDR [6], Blanchet's ProVerif-based work on authentication protocols [7], and later Tamarin studies of TLS 1.3 [8], Signal [9], and WPA2 [10]. Within the IEEE 802 family, the closest related work concerns WPA2's 4-way handshake; however, the MKA key-agreement logic has received comparatively little machine-checked analysis.

Our contribution is therefore complementary. To the best of our knowledge, this is the first machine-verified symbolic analysis of a simplified MKA exchange. The closest methodological precedent is the Tamarin analysis of TLS 1.3 by Cremers et al. [8], which similarly verifies authentication, secrecy, and ordering properties. Our work applies the same general methodology to the Ethernet security domain, where the role of the Key Server introduces a distinct acceptance semantics that is not present in classical protocol examples.

VI. Conclusion

We presented the first (to our knowledge) machine-verified formal security analysis of the MACsec Key Agreement (MKA) protocol using the Tamarin Prover. Our model closely captures the two-party MKA exchange, including the natural-number counter structure for Message Numbers and Key Numbers, and covers both the initial SAK distribution and a subsequent rekey round.

The analysis confirms that the core security goals of MKA hold under the Dolev–Yao adversary model when the pre-shared CAK is not compromised:

- **Agreement and authentication**: the Key Server cannot complete a session with parameters it did not initiate, and the Server must have participated for the session to complete.
- **SAK secrecy**: the adversary cannot learn a session key without compromising the CAK.
- **Session ordering**: rekeying is strictly sequenced after a successful initial session.
- **Key freshness**: each rekeyed SAK is distinct from the preceding one, and its KN advances monotonically.

At the same time, we formally demonstrate a structural limitation: a malicious or compromised Key Server that holds the CAK can inject an arbitrary SAK value that the Server will accept without any means of detection. This reflects MKA's design assumption that the Key Server is fully trusted. Deployments that cannot uphold this trust assumption should consider additional controls, such as cryptographic binding of SAKs to the KDF derivation path through an out-of-band verification mechanism or a distributed key-agreement overlay.

a) Future work.: Several directions remain open:

- Extending the model to multi-party MKA scenarios (multiple CA members beyond two parties).
- Modelling the full IEEE 802.1X-2020 MKA state machine, including the live peer list and SAP/SAI management.
- Analysing the protocol under a partially compromised adversary model where individual session keys (but not the CAK) can be revealed, to study forward-secrecy limitations.
- Investigating whether augmenting MKA with a SAK commitment mechanism, for example a proof of derivation based on the KDF, could prevent malicious Key Server SAK injection.

Our Tamarin model is publicly available at https://github.com/KaplanHalil/MKA_tamarin to support reproducibility and further research.

References

[1] "IEEE standard for local and metropolitan area networks—MAC security (MACsec)," IEEE Std 802.1AE-2018, IEEE, 2018.

[2] "IEEE standard for local and metropolitan area network—port-based network access control," IEEE Std 802.1X-2020, IEEE, 2020.
[3] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN prover for the symbolic analysis of security protocols," in *Computer Aided Verification (CAV 2013)*. Springer, 2013, pp. 696–701.
[4] D. Basin, J. Dreier, and R. Sasse, "Automated verification of security protocols with global state," in *Proceedings of the 22nd ACM Conference on Computer and Communications Security (CCS)*. ACM, 2015, p. I.
[5] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.
[6] G. Lowe, "Breaking and fixing the Needham-Schroeder public-key protocol using FDR," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 1996)*. Springer, 1996, pp. 147–166.
[7] B. Blanchet, "An efficient cryptographic protocol verifier based on Prolog rules," in *14th IEEE Computer Security Foundations Workshop (CSFW)*. IEEE, 2001, pp. 82–96.
[8] C. Cremers, M. Horvat, S. Scott, and T. van der Merwe, "A comprehensive symbolic analysis of TLS 1.3," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1773–1788.
[9] N. Kobeissi, K. Bhargavan, and B. Blanchet, "Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach," in *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2017, pp. 435–450.
[10] M. Vanhoef and F. Piessens, "Key reinstallation attacks: Forcing nonce reuse in WPA2," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1313–1328.